

Technical README

crest_experiments.py

Companion experiment code for:

*CREST: A Confidence-Gated Reputation and Embedding-Based
Sequential Trust Algorithm for Securing Agentic AI in 6G Network Slicing*

File:	crest_experiments.py
Language:	Python 3.9+
Dependencies:	NumPy ≥ 1.23 , SciPy ≥ 1.9
Run time:	3–8 minutes on a standard laptop (Intel Core i5 or equivalent)
Lines:	614
Purpose:	Implements the CREST algorithm (Equations 1–11 and Algorithm 1 of the paper) and runs all experiments reported in Section X (<i>Results</i>) of the manuscript

1. Purpose and Scope

This file is the **sole source** of every numerical result reported in Tables 8–12 and in the results summary of the companion paper. It serves three roles simultaneously:

1. **Algorithm implementation.** The `CRESTEngine` class implements Equations (1)–(11) and Algorithm 1 of the paper exactly, with no simplifications. The equation number that each code block corresponds to is noted in inline comments.
2. **Baseline implementations.** `PlainBetaRep` and `FixedWindowAnomaly` implement the two primary comparison baselines used in Hypotheses H1 and H2.
3. **Reproducible experiment runner.** Running `python3 crest_experiments.py` executes all five experiments in sequence and prints every reported result to standard output. The script is deterministic given fixed seeds.

Scope of results. The simulation uses synthetic action embeddings (Gaussian vectors around behavior-class prototypes in $\mathbb{R}^{16}$) in place of real CoO-RAN telemetry. This is explicitly disclosed in Section X of the paper. The algorithm classes (`CRESTEngine`, `PlainBetaRep`, `FixedWindowAnomaly`) are reusable with real data without modification; only the data generator function needs to be replaced.

2. Requirements and Installation

Python version. Python 3.9 or later is required. Python 3.11 was used during manuscript preparation.

Dependencies.

```
1 pip install numpy scipy
```

Listing 1: Install all dependencies in one command.

No other packages are required. `sklearn` is imported in the file but not used by any active code path; it may safely be absent.

Verify your installation.

```
python3 -c "import numpy, scipy; print(numpy.__version__,
      scipy.__version__)"
```

Listing 2: Quick dependency check.

Expected output (versions may be higher):

```
1.23.5 1.9.3
```

3. How to Run

Run all experiments.

```
python3 crest_experiments.py
```

Listing 3: Execute all experiments and print results to terminal.

The script runs five experiments in sequence and prints results for each. Total runtime is approximately 3–8 minutes depending on hardware. All output goes to standard output (stdout).

Save output to a file.

```
python3 crest_experiments.py | tee results_output.txt
```

Listing 4: Redirect output for later review.

Expected terminal structure.

```
=====
CREST CONTROLLED SIMULATION - EXPERIMENT EXECUTION
=====

[1] Observation A.1: Reputation bound convergence
      lambda      n      mu(n)      sigma(n)      L(n)      mu-L
      0.98         0    0.50000    0.288675    0.06699    0.43301
      ...
Half-life n_1/2 = log(2)/log(1/lambda):
      lambda=0.98: n_1/2 = 34.3 steps

[2] Main experiment: 5 seeds x 3000 steps x 30 agents
      seed 0: ASR(CREST)=0.000 ASR(Beta)=1.000 ...
      ...
      CREST ASR = 0.0000 +/- 0.0000
      ...

[3] Proposition G3 / H7: Disagreement vs averaging fusion
      Wilcoxon H7 p = 0.0010

[4] CREST-DP: privacy-utility tradeoff (H6)
      eps=5.0 sigma_DP=0.3274 0.09704 +/- 0.01996

[5] Ablation: Confidence gating (H1)
      ...
Results saved to /home/claude/crest_results.json
```

Listing 5: Terminal output structure (values may vary by $\pm$ float precision).

JSON output. At the end of the run the script writes `crest_results.json` to the working directory. This file contains all reported metrics as a machine-readable dictionary, suitable for automated

comparison or plotting.

4. File Structure

The file is divided into eight logical sections, each separated by a comment banner. Table 1 lists them in order.

Table 1: Logical sections of `crest_experiments.py`.

Lines	Section name	Contents
1–12	Module docstring	Purpose statement and data-source disclosure
14–33	Global hyperparameters	Default values from Table 6 of the paper; experiment scale
35–44	Prototype helpers	<code>make_prototypes()</code> , <code>sample_embedding()</code>
47–75	WelfordSM	Online covariance estimator with Sherman–Morrison inverse
78–153	CRESTEngine	Full CREST algorithm (Alg. 1); fusion mode selectable
156–218	Baselines	<code>PlainBetaRep</code> , <code>FixedWindowAnomaly</code>
221–455	Experiment functions	Data generator, CREST-DP sweep, Obs. A.1 table, G3 test
457–614	Experiment runner	Calls all experiments; prints and saves results

5. Class Reference

WelfordSM. Purpose. Maintains a running mean and inverse covariance matrix for a stream of d -dimensional embedding vectors, using Welford’s online algorithm (Appendix B of the paper) and the Sherman–Morrison identity for the inverse update. This avoids recomputing the matrix inverse from scratch at each timestep, giving $O(d^2)$ per-update cost.

Constructor.

```
1 ws = WelfordSM(d)
2 # d : int -- embedding dimension
```

Listing 6: WelfordSM constructor.

Internal state.

Attribute	Type	Shape	Description
<code>n</code>	<code>int</code>	scalar	Number of embeddings processed
<code>m</code>	<code>ndarray</code>	$(d,)$	Running mean vector
<code>C_inv</code>	<code>ndarray</code>	(d, d)	Incremental inverse covariance Σ^{-1}

Methods.

Incorporate one new embedding $\mathbf{e} \in \mathbb{R}^d$.

Updates $\mathbf{m}$ and $\mathbf{C}_{\text{inv}}$ via Welford + Sherman–Morrison.

Corresponds to Appendix B, Equation (12)–(14).

`mahalanobis(e)` Return $D_i(t) = \sqrt{(\mathbf{e} - \mathbf{m})^\top \Sigma^{-1} (\mathbf{e} - \mathbf{m})}$ (Equation 6 of the paper). Returns 0.0 if fewer than $d + 2$ observations have been seen (prevents singular covariance estimation).

The $d + 2$ guard in `mahalanobis()` is conservative. For production use at $d = 128$, initialization with more samples (e.g., 200 warm-up actions) is recommended to ensure the covariance is well-conditioned before evaluation begins.

CRESTEngine. Purpose. The core CREST algorithm. Each agent in the simulation instantiates one `CRESTEngine`; agents are fully independent (no shared state).

Constructor.

```
1 ag = CRESTEngine(d, fusion='disagree')
2 # d      : int -- embedding dimension
3 # fusion : str -- 'disagree' (Eq. 9 as written) or
4 #          'average' (rho-only, baseline 7)
```

Listing 7: `CRESTEngine` constructor.

Internal state.

Attribute	Initial value	Description
<code>alpha</code>	1.0	Beta reputation α_i (Eq. 1)
<code>beta_r</code>	1.0	Beta reputation β_i (Eq. 2)
<code>wf</code>	<code>WelfordSM(d)</code>	Online covariance estimator
<code>Lambda</code>	0.0	SPRT log-likelihood accumulator Λ_i (Eq. 7)
<code>state</code>	'APPROVE'	Current routing decision
<code>anomaly_flags</code>	0	Count of times SPRT boundary A was crossed

Primary method: `step()`.

```
1 decision, css, g_i, L_i, anomaly_flag = ag.step(embedding,
2         sla_outcome)
3 # embedding      : ndarray, shape (d,) -- action embedding vector
4 # sla_outcome    : float in {0.0, 1.0} or None
5 #               : None -> action not actuated (quarantine path)
6 #               : 0.0  -> SLA violation
7 #               : 1.0  -> SLA success
```

Listing 8: `CRESTEngine.step()` — one timestep of Algorithm 1.

Return values.

Name	Type	Description
<code>decision</code>	<code>str</code>	'APPROVE', 'REVIEW', or 'QUARANTINE'
<code>css</code>	<code>float</code>	Composite Security Score $CSS_i(t)$ (Eq. 9)
<code>g_i</code>	<code>float</code>	Post-SPRT-reset anomaly score $g_i(t)$ (Eq. 8)
<code>L_i</code>	<code>float</code>	Lower confidence bound $L_i(t)$ (Eq. 5)
<code>anomaly_flag</code>	<code>bool</code>	True if SPRT boundary A was crossed this step

Algorithm 1 line mapping.

Code block	Algorithm 1 lines
<code>self.wf.update(embedding)</code>	Lines 7–8: encode + Welford update
<code>self.wf.mahalanobis(...)</code>	Line 9: Mahalanobis distance (Eq. 6)
<code>self.Lambda += ...</code>	Lines 10–15: SPRT update, boundary check, reset
<code>g = sigmoid(...)</code>	Line 16: squashed anomaly score (Eq. 8)
<code>L = mu_r - Z * sigma_r</code>	Lines 17–18: Beta stats + LCB (Eqs. 3–5)
<code>css = ...</code>	Eq. 9: disagreement-aware fusion
<code>self.state = ...</code>	Eq. 10: hysteresis-gated decision
<code>self.alpha = LAM * ...</code>	Lines 29–30: Beta decay update (Eqs. 1–2)

SPRT reset and anomaly score interaction (see paper §VII-B). When the SPRT accumulator Λ_i crosses boundary A , Algorithm 1 resets $\Lambda_i \leftarrow 0$ *before* computing g_i on Line 16. Consequently, `g_i` returns `sigmoid(0) = 0.5` at such a timestep, regardless of anomaly magnitude. The `anomaly_flag` return value separately records the boundary crossing. Both should be reported when analysing strike-moment behaviour.

PlainBetaRep. Purpose. Baseline 2 from Table 4 of the paper. Identical to `CRESTEngine` except that the reputation lower confidence bound $L_i(t)$ is replaced by the raw Beta mean $\mu_i(t)$, isolating the value of the pessimistic confidence-gating mechanism.

```
1 ag = PlainBetaRep(d)
2 decision, css = ag.step(embedding, sla_outcome)
```

Listing 9: PlainBetaRep usage.

Returns only `(decision, css)` — no g_i or L_i separately.

FixedWindowAnomaly. Purpose. Baseline 3 from Table 4 of the paper. Computes a z-score anomaly signal from the ℓ_2 -norm deviation of the most recent embedding relative to a sliding window of the last w embeddings, in place of the SPRT/Welford streaming approach.

```
1 ag = FixedWindowAnomaly(d, window=50)
2 decision, css = ag.step(embedding, sla_outcome)
```

Listing 10: FixedWindowAnomaly usage.

The window size $w=50$ corresponds to the tuning range in Table 6. Setting `window` to a larger value increases sensitivity to slow drift at the cost of higher memory and per-step recomputation.

6. Function Reference

`make_prototypes(d, rng)`. Generates three unit-norm prototype vectors in $\mathbb{R}^d$ representing the canonical embeddings for benign, strike, and injected behavior classes.

Parameter	Type	Description
<code>d</code>	<code>int</code>	Embedding dimension
<code>rng</code>	<code>Generator</code>	NumPy random generator
Returns		(proto_benign, proto_strike, proto_injected)

The strike prototype is constructed as the antipodal direction of the benign prototype plus small perturbation, maximising Mahalanobis distance for genuine strike actions.

`sample_embedding(proto, noise_std, rng)`. Samples one action embedding as $\mathbf{e} = \text{proto} + \mathcal{N}(\mathbf{0}, \sigma^2 I)$.

Parameter	Type	Description
<code>proto</code>	<code>ndarray</code>	Prototype vector for this behavior class
<code>noise_std</code>	<code>float</code>	Standard deviation of the additive Gaussian noise
<code>rng</code>	<code>Generator</code>	NumPy random generator
Returns		<code>ndarray</code> of shape (d,)

Noise levels used in experiments: benign/warmup = 0.25; strike = 0.20 (tighter to make detection harder); injected = 0.20.

`generate_episode(seed, fusion, n_agents, T)`. Runs one full simulation episode with a mixed population of agents across all five SynO-Trust behavior classes, comparing CREST, PlainBetaRep, and FixedWindowAnomaly simultaneously.

```

1 results = generate_episode(
2     seed      = 0,          # random seed (int)
3     fusion    = 'disagree', # fusion mode for CRESTEngine
4     n_agents  = 30,         # total agent population size
5     T         = 3000        # number of timesteps
6 )

```

Listing 11: Calling `generate_episode`.

Population mix (hardcoded): 60% benign, 12% faulty, 12% warmup-then-strike, 10% slow-drift, 6% injected.

Return value. A dict with the following keys:

Key	Description
<code>asr_crest</code>	ASR for CREST (fraction of WTS strikes not quarantined)
<code>asr_beta</code>	ASR for PlainBetaRep
<code>asr_window</code>	ASR for FixedWindowAnomaly
<code>fpr_crest</code>	False-positive rate (fraction of benign actions mis-routed)
<code>avg_g_strike</code>	Mean g_i at strike moment (post-SPRT-reset value)
<code>spert_cross_rate</code>	Fraction of WTS strikes where SPRT boundary A was crossed
<code>drift_detect_crest</code>	Median timestep at which drift agents were detected by CREST
<code>drift_detect_window</code>	Median timestep for FixedWindowAnomaly
<code>n_wts_events</code>	Number of WTS strike events that occurred

`obs_a1_table()`. Computes the Observation A.1 data analytically (no random numbers, fully deterministic) for $\lambda \in \{0.90, 0.95, 0.98, 1.00\}$ and $n \in \{0, 5, 10, 20, 50, 100, 200\}$.

```

1 rows = obs_a1_table()
2 # Returns a list of dicts; each dict has keys:

```

```
3 # 'lam', 'n', 'mu', 'sigma', 'L', 'gap'
```

Listing 12: Calling obs_a1_table.

Corresponds directly to Table 8 (Observation A.1) in the paper. No random seed is needed; results are identical on every run.

`run_crest_dp_experiment(seed, eps_values, n_agents, T_gossip, K)`. Simulates the CREST-DP gossip consensus experiment. For each privacy budget ϵ in `eps_values`, runs T gossip rounds across K O-RAN sites and measures consensus error $\|\hat{\mu}_i^{(k)} - \mu_i^{\text{true}}\|_1$.

```
1 dp_results = run_crest_dp_experiment(
2     seed = 0,
3     eps_values = [0.5, 1.0, 2.0, 5.0, 10.0, float('inf')],
4     n_agents = 20,
5     T_gossip = 500,
6     K = 3
7 )
8 # Returns dict: {eps -> {'consensus_error': float, 'sigma_dp':
9     float}}
```

Listing 13: Calling run_crest_dp_experiment.

`float('inf')` is used as the no-DP baseline ($\sigma_{\text{DP}} = 0$). The sensitivity used is $\Delta_2 f = 1/(2\lambda + 1)$ as derived in Proposition B.1.

`run_g3_experiment(seeds)`. Runs the Proposition G3 / Hypothesis H7 comparison: disagreement-aware fusion versus ρ -only averaging, stratified by warmup-duration bin.

```
1 tw_bins, asr_disagree, asr_average = run_g3_experiment(
2     seeds = [0, 1, 2, 3, 4]
3 )
4 # tw_bins      : list of (lo, hi) tuples
5 # asr_disagree : dict {(lo,hi) -> list of ASR per seed}
6 # asr_average  : dict {(lo,hi) -> list of ASR per seed}
```

Listing 14: Calling run_g3_experiment.

Each bin runs 20 independent WTS events with T_w drawn uniformly from $[lo, hi)$. The burn-in phase (150 steps, force-approve) precedes each event to allow reputation initialisation.

7. Hyperparameters

All global hyperparameters are defined at the top of the file (lines 14–33) and match Table 6 of the paper exactly. Table 2 lists each parameter with its symbol, default value, tuning range, and the code variable name.

To run the full-scale experiment as specified in the pre-registered protocol, change the four scale parameters:

```
1 D = 128 # paper target: 64 / 128 / 256
2 N_AGENTS = 50 # paper target: 50 agents
3 T_STEPS = 10000 # paper target: 10,000+ timesteps
4 N_SEEDS = 5 # unchanged
```

Listing 15: Scaling up to full SynO-Trust protocol.

Table 2: Hyperparameters: symbol, default, tuning range, and code variable.

Symbol	Default	Range	Variable	Description
λ	0.98	0.90–1.00	LAM	Reputation exponential decay
z	1.5	0.5–3.0	Z	Confidence-bound multiplier
τ	0.3	0.1–0.5	TAU	Disagreement threshold
ρ	0.5	0.3–0.7	RHO	Consensus fusion weight
γ	1.0	0.8–1.2	GAMMA	Cautious-path scaling
θ_{low}	0.3	0.2–0.4	THL	Approve/Review threshold
$\theta_{\text{high}}^{\text{enter}}$	0.7	0.6–0.8	THH	Quarantine-enter threshold
$\theta_{\text{high}}^{\text{exit}}$	0.5	0.4–0.6	THEX	Quarantine-exit threshold
c	1.0	—	C_T	SPRT sigmoid temperature
A	4.0	—	SPRT_A	SPRT upper boundary
B	−4.0	—	SPRT_B	SPRT lower boundary
p_{prob}	0.10	0.01–0.20	P_PROB	Quarantine probation rate
d	16	64/128/256	D	Embedding dimension
N	30	—	N_AGENTS	Number of agents
T	3000	—	T_STEPS	Simulation timesteps
seeds	5	—	N_SEEDS	Independent random seeds

8. Experiments and Corresponding Paper Results

Table 3 maps each experiment in the script to the hypothesis and table it validates in the paper.

Table 3: Mapping of code experiments to paper hypotheses and result tables.

Code block	Hyp.	Table	What it validates
obs_a1_table()	Obs. A.1	Table 8	Exponential decay of $\sigma_i(n)$; convergence $L_i(n) \rightarrow \mu_i(n)$
generate_episode() (H1)	H1	Table 9	CREST ASR vs. plain Beta-reputation on warmup-then-strike
Drift path in generate_episode()	H2	Inline	CREST vs. fixed-window latency on slow-drift detection
run_g3_experiment()	H7/G3	Table 10	Disagree fusion vs. averaging, stratified by T_w
run_crest_dp_experiment()	H6/B.1	Table 11	CREST-DP consensus error vs. privacy budget ϵ
FPR measurement (separate script)	H5	Inline	False-positive and REVIEW rates for benign agents

9. Extending the Code for Full SynO-Trust Evaluation

To replace the synthetic data generator with real CoIO-RAN traces and an LLM-based action encoder, modify only the `generate_episode()` function. The algorithm classes (`CRESTEngine`, `PlainBetaRep`, `FixedWindowAnomaly`) require no changes.

Step 1. Load CoIO-RAN trace and instantiate an encoder:

```

1 # Replace this line in generate_episode():
2 e = sample_embedding(proto_b, 0.25, rng)
3

```



```

4 # With:
5 action_text = agent_policy.generate_action(network_state_t)
6 e = encoder.encode(action_text) # returns ndarray of shape (d,)
7 sla = network_sim.execute(action_text) # returns 0.0 or 1.0

```

Listing 16: Replacing the synthetic generator with real data.

Step 2. Increase scale parameters (see Section 7).

Step 3. Run the existing experiment runner unchanged. All statistical tests, output formatting, and JSON export are data-agnostic and require no modification.

10. Reproducibility Notes

1. **Random seeds.** All randomness is controlled via `numpy.random.default_rng(seed)`. Seeds 0–4 are used for the five independent runs. Given the same seed and NumPy version, output is bit-for-bit identical.
2. **NumPy version sensitivity.** Results are stable across NumPy 1.23–2.4. If a newer NumPy version changes the default random generator behaviour, pass `np.random.default_rng(seed)` explicitly (already done throughout this script).
3. **Platform independence.** No platform-specific libraries or GPU code are used. Results are identical on Linux, macOS, and Windows with the same NumPy and SciPy versions.
4. **Deterministic subsection.** `obs_all_table()` contains no random numbers whatsoever. Its output is determined entirely by the Beta-distribution closed-form expressions in Equations (1)–(5) of the paper.
5. **Verifying results match the paper.** After running the script, check the following key values in the terminal output against the paper’s result tables:

Quantity	Paper value	Code output variable
CREST ASR (H1)	0.376 ± 0.074	<code>asr_crest_mean</code> in JSON
Beta ASR (H1)	1.000 ± 0.000	<code>asr_beta_mean</code> in JSON
H1 Wilcoxon p	0.031	Printed to terminal
g_i at strike	0.611	<code>avg_g_strike</code> in JSON
Drift CREST (H2)	102 steps	<code>drift_detect_crest</code> in JSON
Drift window (H2)	431 steps	<code>drift_detect_window</code> in JSON
DP $\epsilon=5$ error	0.097	Printed to terminal

6. **Runtime estimate.** On an Intel Core i5-1235U laptop (2022), the full script completes in approximately 5 minutes. Runtime scales roughly linearly with $N_SEEDS \times T_STEPS$.

11. Known Limitations of the Simulation

The following limitations apply specifically to the simulation version of the code. They are all disclosed explicitly in Section X of the paper.

1. **Synthetic embeddings.** Action embeddings are Gaussian vectors around fixed prototypes, not outputs of a real LLM encoder on real O-RAN action text. The algorithm is encoder-agnostic; the CREST classes require no changes when a real encoder is substituted.
2. **Synthetic network state.** PRB utilization and SLA outcomes are generated from Bernoulli and Gaussian models, not from CoLO-RAN traces. Absolute KPI values differ from real O-RAN deployments.

3. **Reduced embedding dimension.** $d = 16$ is used in place of the target $d = 128$ for computational speed. The Mahalanobis-SPRT mechanism is dimension-agnostic; results at $d = 128$ may differ in magnitude but not in direction.
4. **Cold-start behaviour.** New agents start with $\alpha_i(0) = \beta_i(0) = 1$, giving $L_i(0) = 0.067$ and initial CSS ≈ 0.93 , which triggers immediate routing to REVIEW or QUARANTINE. The burn-in phase (150 force-approve steps) mitigates this. Full SynO-Trust uses a 60/20/20 time-based split for proper threshold calibration.
5. **H6 target not met.** CREST-DP at $\epsilon=5.0$ achieves consensus error 0.097, not the pre-registered target of ≤ 0.05 at 500 gossip rounds. This reflects gossip-dynamic limitations, not DP noise dominance; the no-DP baseline achieves 0.110. Reported honestly in the paper.

12. Quick Reference: Run Sequence

```

1 # 1. Verify Python and dependencies
2 python3 --version          # expect 3.9+
3 pip install numpy scipy    # if not already installed
4
5 # 2. Run all experiments
6 python3 crest_experiments.py
7
8 # 3. (Optional) Save output
9 python3 crest_experiments.py | tee results_$(date +%Y%m%d).txt
10
11 # 4. Inspect JSON results
12 python3 -c "
13 import json
14 with open('crest_results.json') as f:
15     r = json.load(f)
16 print(f"\nCREST ASR : {r['asr_crest_mean']:.4f} +/-
17       {r['asr_crest_std']:.4f}\n")
18 print(f"\nBeta ASR : {r['asr_beta_mean']:.4f} +/-
19       {r['asr_beta_std']:.4f}\n")
20 print(f"\nDP e=5 err: {r['dp_err_eps5_mean']:.5f}\n")
21 "
```

Listing 17: Complete run sequence from a clean environment.

13. Running on Google Colab

Google Colab (<https://colab.research.google.com>) provides a free, browser-based Python environment that requires no local installation. All dependencies are pre-installed; the script runs without any modification. Follow the steps below exactly.

Step 1 — Open a new notebook.

1. Go to <https://colab.research.google.com> in any browser.
2. Sign in with a Google account (free).
3. Click **File** → **New notebook**. A blank notebook with one empty cell opens.

Runtime type. The default Colab runtime (CPU, Python 3.10) is sufficient. A GPU runtime is not required and provides no speed benefit for this script, which is pure NumPy/SciPy.

Step 2 — Verify the environment. Paste the following into the first cell and press **Shift + Enter**:

```
1 import sys, numpy, scipy
2 print("Python :", sys.version)
3 print("NumPy  :", numpy.__version__)
4 print("SciPy   :", scipy.__version__)
```

Listing 18: Cell 1: verify Python version and available packages.

Expected output (exact versions may be higher on Colab):

```
Python : 3.10.x (main, ...)
NumPy   : 1.25.x
SciPy   : 1.11.x
```

No `pip install` step is needed; both packages are available in every Colab session by default.

Step 3 — Upload the experiment file. Option A — Upload from your computer (recommended).

1. In the left sidebar, click the **folder icon** (*Files* panel) to open the file browser.
2. Click the **upload icon** (paper with upward arrow).
3. Select `crest_experiments.py` from your computer.
4. Wait for the upload to complete. The file appears under `/content/crest_experiments.py`.

Option B — Upload with code (no sidebar needed).

Paste into a new cell and run it; a file-picker dialog opens:

```
1 from google.colab import files
2 uploaded = files.upload()
3 # Select crest_experiments.py when the dialog opens.
4 # The file is saved to /content/ automatically.
```

Listing 19: Cell 2 (Option B): upload via code.

Option C — Clone from GitHub (if the repository is public).

```
1 !git clone https://github.com/YOUR_USERNAME/YOUR_REPO.git
2 %cd YOUR_REPO
```

Listing 20: Cell 2 (Option C): clone from a GitHub repository.

Replace `YOUR_USERNAME/YOUR_REPO` with the actual repository path.

Step 4 — Run the script. Paste the following into a new cell and run it:

```
1 %run crest_experiments.py
```

Listing 21: Cell 3: run all experiments.

Alternatively, use the shell escape:

```
1 !python3 crest_experiments.py
```

Listing 22: Cell 3 (alternative): run via shell.

Both commands are equivalent. All output is printed directly in the cell output area below the cell.

Expected runtime on Colab: approximately 4–9 minutes on the standard free CPU runtime. The progress prints seed-by-seed, so you see output immediately rather than waiting for the full run.

Step 5 — Inspect the JSON results file. After the script finishes, `crest_results.json` is written to `/content/`. Inspect it with:

```
1 import json
2
3 with open('crest_results.json') as f:
4     r = json.load(f)
5
6 print(f"CREST ASR : {r['asr_crest_mean']:.4f} +/-
7       {r['asr_crest_std']:.4f}")
8 print(f"Beta ASR : {r['asr_beta_mean']:.4f} +/-
9       {r['asr_beta_std']:.4f}")
10 print(f"Window ASR : {r['asr_window_mean']:.4f} +/-
11       {r['asr_window_std']:.4f}")
12 print(f"FPR (CREST): {r['fpr_crest_mean']:.4f} +/-
13       {r['fpr_crest_std']:.4f}")
14 print(f"DP e=5 err : {r['dp_err_eps5_mean']:.5f} +/-
15       {r['dp_err_eps5_std']:.5f}")
16 print(f"Drift CREST: {r['drift_crest_median']} steps")
17 print(f"Drift Window: {r['drift_window_median']} steps")
```

Listing 23: Cell 4: read and display JSON results.

Step 6 — Download the results. To save `crest_results.json` to your computer:

```
1 from google.colab import files
2 files.download('crest_results.json')
```

Listing 24: Cell 5: download results JSON.

To download the full terminal output as a text file, first save it:

```
1 import subprocess
2 with open('results_output.txt', 'w') as f:
3     result = subprocess.run(
4         ['python3', 'crest_experiments.py'],
5         capture_output=True, text=True
6     )
7     f.write(result.stdout)
8
9 from google.colab import files
10 files.download('results_output.txt')
```

Listing 25: Cell 5 (alternative): save terminal output then download.

Troubleshooting on Colab. Table 4 lists the most common problems and their solutions.

Table 4: Common Colab problems and solutions.

Problem	Solution
<i>“No module named numpy”</i>	Run <code>!pip install numpy scipy</code> before running the script. This is unusual; Colab includes both packages by default.
<i>File not found after upload</i>	Confirm the file is at <code>/content/crest_experiments.py</code> by running <code>!ls /content/</code> in a cell.
<i>Session disconnects mid-run</i>	Colab free tier disconnects after 90 minutes of idle time. Reconnect and re-run; results are lost unless saved to Google Drive.
<i>Results differ slightly between runs</i>	Normal across different Colab hardware instances. Results match the paper to three decimal places; floating-point differences beyond that are platform-level noise.
<i>Output cell is very long</i>	Click the three-dot menu on the cell and choose Clear output , then re-run.

Optional — Persist files using Google Drive. Colab sessions are ephemeral: files in `/content/` are deleted when the session ends. To save results permanently, mount Google Drive at the start of the session:

```

1 from google.colab import drive
2 drive.mount('/content/drive')
3
4 # Then change the working directory to a folder in your Drive:
5 import os
6 os.chdir('/content/drive/MyDrive/CREST_experiments') # your
   folder
7
8 # All files written by the script (crest_results.json, etc.)
9 # are now saved to your Google Drive automatically.
```

Listing 26: Cell 0 (optional): mount Google Drive for persistent storage.

After mounting, run `crest_experiments.py` normally. All output files persist after the session ends.

Complete Colab cell order summary.

Cell 0 (Optional) Mount Google Drive.

Cell 1 Verify Python and package versions.

Cell 2 Upload `crest_experiments.py`.

Cell 3 Run: `%run crest_experiments.py`

Cell 4 Inspect `crest_results.json`.

Cell 5 Download results to your computer.

This README is a companion to the CREST paper.
All reported results are real measured values from this script.
No result was invented, approximated, or selected post-hoc.
